

BANCO DE DADOS II

Atividade Prática

Modelagem e Implementação de um Sistema de Agendamento de Manutenção Veicular

Disciplina: Banco de Dados II | Curso: Bacharelado em Sistemas de Informação

1. Contexto do Problema

Uma oficina mecânica de médio porte localizada em Natal/RN atende aproximadamente 80 veículos por dia. Todo o controle operacional, clientes, veículos, agendamentos, serviços, peças e pagamentos, é feito em planilhas e anotações físicas, gerando retrabalho, perda de histórico e impossibilidade de análise gerencial.

O proprietário contratou sua equipe para projetar e implementar um banco de dados relacional em PostgreSQL capaz de suportar as operações da oficina com integridade, desempenho e escalabilidade suficientes para o crescimento dos próximos cinco anos.

Você deverá realizar as três etapas clássicas do ciclo de projeto de banco de dados: modelagem lógica, implementação física e população com dados representativos.

2. Requisitos do Sistema

2.1 Requisitos Funcionais

- O sistema deve permitir o cadastro de clientes, diferenciando pessoa física (CPF) e pessoa jurídica (CNPJ).
- Cada cliente pode possuir um ou mais veículos registrados.
- Cada veículo pode ter múltiplos agendamentos ao longo do tempo.
- Um agendamento (ordem de serviço) deve registrar quais serviços foram executados e quais peças foram consumidas.
- Cada serviço executado deve estar vinculado a um funcionário responsável.
- O sistema deve controlar o estoque de peças, incluindo quantidade atual e quantidade mínima.
- Cada ordem de serviço concluída deve permitir o registro do pagamento, com suporte a múltiplas formas de pagamento e parcelamento.
- Clientes devem poder registrar uma avaliação (nota de 1 a 5 e comentário) para cada agendamento concluído.
- O sistema deve armazenar o hodômetro (quilometragem) na entrada e na saída do veículo a cada visita.

2.2 Regras de Negócio

As regras a seguir são obrigatórias e devem ser implementadas como restrições no banco de dados, não na aplicação.

- Um cliente Pessoa Física deve ter CPF preenchido e CNPJ nulo; um cliente Pessoa Jurídica deve ter CNPJ preenchido e CPF nulo. Ambos não podem ser nulos simultaneamente.
- CPF e CNPJ são únicos no sistema — não podem existir dois cadastros com o mesmo documento.
- O endereço de e-mail de um cliente deve ser único.
- A data/hora de conclusão de um agendamento não pode ser anterior à data/hora de abertura.
- A quilometragem de saída não pode ser inferior à quilometragem de entrada no mesmo agendamento.
- O estoque de uma peça não pode assumir valor negativo.
- O valor total de uma ordem de serviço é a soma do total de mão de obra e o total de peças; esse campo deve ser mantido automaticamente pelo banco.
- O total de cada item (serviço ou peça) deve ser calculado automaticamente a partir de quantidade, preço unitário e percentual de desconto.
- Cada agendamento pode ter no máximo uma avaliação associada.
- Somente agendamentos com status 'Concluído' podem receber pagamento ou avaliação.
- O salário de um funcionário deve ser maior que zero.
- A nota de avaliação deve estar no intervalo de 1 a 5.

2.3 Requisitos Não Funcionais

- O banco de dados deve ser implementado em PostgreSQL versão 17 ou superior.
- O script SQL completo deve ser idempotente: deve poder ser executado múltiplas vezes sem gerar erros (use DROP ... IF EXISTS antes das criações).
- Todos os campos monetários devem utilizar o tipo NUMERIC com precisão adequada. O uso de FLOAT ou REAL é proibido para valores financeiros.
- Campos de data e hora devem utilizar TIMESTAMPTZ ou DATE, conforme a semântica de cada campo.
- O banco deve estar organizado em um schema dedicado, separado do schema público.
- Tipos de dados recorrentes (CPF, CNPJ, placa, e-mail, telefone) devem ser definidos como domínios reutilizáveis (CREATE DOMAIN).
- Todos os índices criados devem ser acompanhados de um comentário SQL que justifique sua existência em termos da consulta que acelera.
- Cada índice criado deve ser decisão fundamentada em plano de execução coletado com EXPLAIN ANALYZE — não em suposição. Ver Seção 5.

3. Entidades Esperadas (Escopo Essencial)

O modelo deve conter, no mínimo, as seguintes entidades. Atributos, tipos de dados, nomes de colunas e restrições são de responsabilidade do aluno e fazem parte da avaliação.

Entidade	Descrição do papel no sistema
cliente	Pessoa física ou jurídica que traz o veículo para manutenção
veiculo	Automóvel vinculado a um cliente; pode ter múltiplos agendamentos
funcionario	Mecânico, eletricista, atendente ou gestor da oficina
tipo_servico	Catálogo de serviços oferecidos (troca de óleo, freios, A/C etc.)

peca	Item de estoque utilizado durante a execução de um serviço
agendamento	Ordem de serviço que registra a visita de um veículo à oficina
item_servico	Associação entre um agendamento e os serviços realizados
item_peca	Associação entre um agendamento e as peças consumidas
pagamento	Registro financeiro de quitação de uma ordem de serviço
avaliacao	Nota e comentário do cliente sobre o atendimento recebido

Observações sobre o modelo:

- As entidades item_servico e item_peca são tabelas associativas que resolvem relacionamentos muitos-para-muitos. Certifique-se de que cada uma carregue os atributos corretos de relacionamento (quantidades, preços, descontos).
- Entidades adicionais que o aluno julgue necessárias para atender algum requisito são permitidas, desde que justificadas no relatório.
- Atributos calculados (totais, somas) devem ser implementados como colunas geradas (GENERATED ALWAYS AS ... STORED) — nunca armazenados manualmente.

4. Entregas Obrigatórias

4.1 Entrega 1 — Modelo Lógico

Produza o Diagrama Entidade-Relacionamento (DER) ou o Modelo Relacional (MR) do sistema, contendo:

- Todas as entidades com seus atributos, identificando chave primária, chave estrangeira e atributos simples.
- Cardinalidades de todos os relacionamentos, com participação total ou parcial devidamente indicada.
- Representação explícita das principais restrições de integridade (unicidade, obrigatoriedade, domínios).
- Ferramentas aceitas: brModelo, dbdiagram.io, MySQL Workbench, draw.io, ou diagrama elaborado manualmente e digitalizado com legibilidade.
- Formato de entrega: imagem PNG/JPG ou PDF de alta resolução.

4.2 Entrega 2 — Modelo Físico (DDL)

Produza um script SQL executável no PostgreSQL contendo:

- Criação do schema dedicado e configuração do search_path.
- Definição dos domínios (CREATE DOMAIN) para os tipos recorrentes listados nos requisitos não funcionais.
- CREATE TABLE para todas as entidades, com: tipos de dados adequados, chaves primárias e estrangeiras, constraints CHECK para todas as regras de negócio, valores DEFAULT apropriados, colunas GENERATED para totais calculados.
- Neste script não devem constar CREATE INDEX. Os índices são definidos na Entrega 4, após o processo de análise com EXPLAIN ANALYZE.
- O script deve estar organizado, comentado e pronto para execução sem erros em um banco PostgreSQL limpo.

4.3 Entrega 3 — Carga de Dados

Produza um script SQL separado que popule o banco com volumes mínimos conforme a tabela abaixo. Os dados inseridos devem ser coerentes entre si: chaves estrangeiras válidas, valores dentro dos domínios definidos, ausência de duplicidade em campos únicos.

Atenção: o uso de `generate_series()` e funções de geração automática do PostgreSQL é recomendado para atingir os volumes exigidos. Dados gerados mecanicamente são aceitos, desde que respeitados os tipos e restrições do modelo físico.

Tabela	Mínimo de registros	Restrições de conteúdo exigidas
cliente	200	Deve incluir pessoa física (CPF) e pessoa jurídica (CNPJ)
veiculo	500	Distribuídos entre os clientes cadastrados
funcionario	50	Cargos e especialidades variados
tipo_servico	20	Serviços distintos com preços e tempos estimados
peca	40	Estoque variado, incluindo itens abaixo do mínimo
agendamento	3.500	Status diversificados: Agendado, Em andamento, Concluído, Cancelado, No-show
item_servico	7.000	Pelo menos 2 serviços por agendamento em média
item_peca	5.000	Distribuídos pelos agendamentos com dados coerentes
pagamento	3.000	Apenas para agendamentos com status Concluído
avaliacao	2.200	Notas variadas; somente agendamentos concluídos podem ser avaliados

Volume total mínimo esperado: aproximadamente 22.100 registros distribuídos entre as tabelas.

4.4 Entrega 4 — Consultas Analíticas

Implemente as oito consultas SQL listadas abaixo. Para cada consulta, o relatório deve conter: (a) o código SQL executado, (b) captura de tela ou exportação do resultado, e (c) uma análise textual de 3 a 5 linhas interpretando o resultado obtido com os dados populados.

1. Contagem de registros por tabela — verifique se os volumes mínimos foram atingidos.
2. Receita total e ticket médio por mês, considerando apenas os agendamentos com status 'Concluído', para os últimos 12 meses.
3. Ranking dos 10 tipos de serviço mais realizados, ordenados por quantidade de execuções e pelo faturamento total acumulado.
4. Ranking de funcionários por faturamento gerado em ordens de serviço concluídas, exibindo também a quantidade de OS atendidas.
5. Listagem dos 20 clientes com maior gasto acumulado, distinguindo pessoa física de jurídica.
6. Distribuição percentual das formas de pagamento confirmadas, exibindo quantidade de transações e valor total por modalidade.
7. Listagem de peças com estoque atual abaixo do estoque mínimo, exibindo o déficit calculado e o fornecedor.
8. Nota média de avaliação por funcionário, considerando apenas funcionários com ao menos cinco avaliações registradas.

5. Entrega 4 — Processo de Otimização com EXPLAIN ANALYZE

Esta entrega é o núcleo técnico da atividade. O objetivo não é apenas criar índices — é demonstrar um ciclo de decisão baseado em evidências empíricas, usando o plano de execução do PostgreSQL como instrumento de diagnóstico e validação.

Nenhum índice deve ser criado por intuição ou convenção. Cada CREATE INDEX deve ser precedido de uma coleta de plano sem o índice e sucedido de uma coleta de plano com o índice, ambas documentadas. Índices sem esse par de evidências não serão considerados válidos na avaliação.

5.1 O que é EXPLAIN ANALYZE e como usá-lo

EXPLAIN mostra o plano que o otimizador de consultas do PostgreSQL pretende executar. EXPLAIN ANALYZE executa a consulta de fato e retorna tanto o plano estimado quanto as métricas reais de execução: tempo em milissegundos, número de linhas processadas por nó do plano, loops e uso de buffers.

Para cada consulta analisada, use obrigatoriamente a forma completa:

- EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) seguido da consulta.
- Execute o comando ANALYZE na tabela antes da primeira coleta, para garantir que as estatísticas do otimizador estão atualizadas.
- Execute cada consulta ao menos três vezes e registre o menor tempo obtido, para eliminar efeitos de cache frio.

5.2 Como interpretar o plano de execução

O plano de execução é uma árvore de nós. Cada nó representa uma operação: varredura de tabela, junção, ordenação, agregação. Os elementos mais importantes a observar são:

Elemento do plano	O que revela e quando é um problema
Seq Scan	Varredura sequencial: lê todas as linhas da tabela. Aceitável em tabelas pequenas ou quando a seletividade é baixa (>20% das linhas). Em tabelas grandes com filtros seletivos, indica ausência de índice útil.
Index Scan	Usa um índice para localizar linhas específicas. Eficiente para alta seletividade. O campo Index Cond mostra qual predicado foi satisfeito pelo índice.
Index Only Scan	Resolve a consulta inteiramente pelo índice, sem tocar a tabela. Mais eficiente que Index Scan quando todas as colunas projetadas estão no índice.
Bitmap Heap Scan	Combinação de bitmap de índice com acesso à tabela. Aparece quando múltiplos índices são combinados ou quando a seletividade é média.
Hash Join / Nested Loop / Merge Join	Estratégias de junção. Nested Loop é eficiente para tabelas pequenas ou com índice na coluna de junção. Hash Join consome memória mas é eficiente para grandes volumes sem índice.
rows=N (estimated) vs rows=N (actual)	Divergência grande entre estimativa e realidade indica estatísticas desatualizadas (execute ANALYZE) ou distribuição de dados irregular.
actual time=X..Y ms	Tempo real de execução do nó. O valor Y é o tempo até a última linha. Compare o nó mais caro antes e depois do índice.
Buffers: shared hit=N read=M	hit = leitura de cache (rápida); read = leitura de disco (lenta). Um índice eficiente reduz drasticamente o valor de read.

5.3 Consultas que devem ser analisadas

As oito consultas abaixo devem ser implementadas e submetidas ao ciclo completo de análise descrito na Seção 5.4. Para cada uma, o enunciado indica a operação esperada — o SQL é responsabilidade do aluno.

9. Contagem de registros por tabela — verificação de volume.
10. Receita total e ticket médio por mês, para os últimos 12 meses, considerando apenas agendamentos com status Concluído.
11. Ranking dos 10 tipos de serviço mais realizados, ordenados por quantidade de execuções e pelo faturamento total acumulado.
12. Ranking de funcionários por faturamento gerado em ordens de serviço concluídas, com quantidade de OS atendidas.
13. Listagem dos 20 clientes com maior gasto acumulado, distinguindo pessoa física de jurídica.
14. Distribuição percentual das formas de pagamento confirmadas, com quantidade de transações e valor total por modalidade.
15. Listagem de peças com estoque atual abaixo do estoque mínimo, exibindo o déficit calculado e o fornecedor.
16. Nota média de avaliação por funcionário, restrita a funcionários com ao menos cinco avaliações registradas.

As consultas 2, 3, 4 e 5 envolvem filtros sobre colunas de alta frequência de acesso (data, status, FK) e junções entre tabelas de grande volume. Espera-se que o aluno identifique, por meio dos planos coletados, quais colunas são candidatas a índice e valide empiricamente o ganho obtido.

5.4 Ciclo obrigatório de análise por consulta

Para cada uma das oito consultas, o relatório deve conter os seguintes elementos, nesta ordem:

- Identificação da consulta (número e descrição).
- SQL executado.
- Plano de execução sem índices adicionais — saída completa do EXPLAIN ANALYZE, colada como texto.
- Diagnóstico: identifique o nó mais custoso, o tipo de varredura utilizado, o tempo total e a divergência entre linhas estimadas e reais.
- Decisão: com base no diagnóstico, descreva se um índice é justificado, em qual coluna e por quê. Se a conclusão for que nenhum índice é necessário, justifique essa decisão também.
- Script CREATE INDEX resultante da decisão (quando aplicável), com comentário embutido que reproduz o argumento do diagnóstico em uma linha.
- Plano de execução após a criação do índice — nova saída do EXPLAIN ANALYZE.
- Comparação quantitativa: preencha a tabela abaixo para cada consulta analisada.

Métrica	Sem índice	Com índice	Redução (%)
Tempo total (ms)	—	—	—
Tipo de varredura	—	—	—
Linhas lidas (actual rows)	—	—	—
Buffers read (disco)	—	—	—

Observação: se a consulta não se beneficiar de índice (ex.: varredura completa é a estratégia ótima dada a seletividade), a tabela deve ser preenchida com os valores iguais e a justificativa deve explicar por que o Seq Scan é a escolha correta do otimizador neste caso.

5.5 Síntese dos índices criados

Ao final do processo, o relatório deve conter uma tabela consolidando todos os índices criados:

Índice	Tabela	Coluna(s)	Consulta que motivou	Redução de tempo
(preencher)				

Índices duplicados ou não documentados por evidência de EXPLAIN ANALYZE serão desconsiderados na avaliação.

5.6 Script final consolidado de índices

Entregue um arquivo separado (05_indices.sql) contendo apenas os CREATE INDEX resultantes do processo de análise, organizados por tabela, cada um precedido de um comentário com: a consulta que motivou sua criação e a redução de tempo observada.

6. Critérios de Avaliação

Critério	Peso	Descritores de qualidade
Modelo Lógico (MER/DLR)	20%	Entidades, cardinalidades, atributos, restrições
Modelo Físico — Script DDL	20%	Tipos de dados, constraints, domínios
Volume e coerência dos dados	15%	Volumes mínimos atingidos, dados sem inconsistências
Processo de otimização (EXPLAIN ANALYZE)	25%	Ciclo antes/depois documentado, índices justificados por evidência empírica, análise do plano de execução
Consultas SQL analíticas	10%	Correção e clareza das queries
Documentação e organização	10%	README, comentários no SQL, nomenclatura consistente

Penalizações automáticas:

- Script DDL não executável sem modificação: desconto de 40% na nota da Entrega 2.
- Volume de dados abaixo do mínimo em qualquer tabela: desconto de 15% na nota da Entrega 3.
- Dados incoerentes (FK inválida, violação de domínio, valores absurdos): desconto proporcional à gravidade.
- Índice criado sem par de planos EXPLAIN ANALYZE (antes e depois): índice não contabilizado na avaliação.
- Ausência de diagnóstico ou comparação quantitativa em qualquer consulta da Entrega 4: zero no subitem correspondente.
- Plágio identificado entre trabalhos ou de fontes externas: nota zero para todos os envolvidos.

7. Orientações Técnicas

Projeto do modelo

- Identifique os relacionamentos N:M antes de escrever SQL. Relacionamentos N:M sempre exigem tabela associativa com atributos próprios.
- Nomeie as constraints explicitamente para facilitar a identificação em mensagens de erro.
- Prefira chaves surrogate (SERIAL ou GENERATED ALWAYS AS IDENTITY) para PKs. Evite usar documentos como PK — use-os como UNIQUE.

Tipos de dados e integridade

- Use NUMERIC(precisão, escala) para valores monetários. FLOAT acumula erro de ponto flutuante.
- Implemente todas as regras de negócio da Seção 2.2 como constraints no banco, não na aplicação.
- Teste cada constraint: insira registros que a violem e confirme que o banco rejeita a operação.

Carga de dados e realismo

- Valide as FKs ao popular: insira primeiro as tabelas-pai, depois as tabelas-filho.
- Ao usar generate_series(), garanta que os valores gerados para FKs existam na tabela referenciada.
- Inclua dados com todos os status possíveis no agendamento — Cancelado e No-show incluídos — para que as consultas gerenciais sejam realistas.

Processo EXPLAIN ANALYZE

- Execute ANALYZE na tabela imediatamente antes de coletar o plano, para garantir estatísticas atualizadas.
- Desative o cache de resultado entre as coletas para evitar leituras do buffer pool: reinicie a conexão ou use pg_prewarm para simular cold cache quando necessário.
- Um índice em coluna de baixa cardinalidade (ex.: status com 5 valores distintos) frequentemente não é utilizado pelo otimizador em tabelas grandes — documente essa observação se ocorrer.
- Índices compostos só são vantajosos quando o predicado filtra por todas as colunas na ordem do índice. Um índice em (status, data_hora) não acelera filtros apenas por data_hora.
- Após criar todos os índices, verifique se algum índice tornou-se redundante. Documente índices descartados por redundância na síntese.

8. Formato e Prazo de Entrega

8.1 Estrutura do arquivo entregue

Submeta um único arquivo .zip nomeado como: BD_[MATRÍCULA]_[NOME_SOBRENOME].zip

O zip deve conter exatamente os seguintes arquivos:

- 01_modelo_logico.pdf (ou .png) — diagrama MER/DLR legível
- 02_ddl.sql — criação do schema, domínios e tabelas, sem índices
- 03_dados.sql — script de população das tabelas
- 04_consultas.sql — as oito consultas analíticas
- 05_indices.sql — script final consolidado de CREATE INDEX com comentários

- 06_relatorio.pdf — ciclo EXPLAIN ANALYZE completo, comparações quantitativas e síntese dos índices
- README.txt — ordem de execução dos scripts, versão do PostgreSQL utilizada e tempo total aproximado para execução da carga

8.2 Prazo e canal de entrega

- Entrega exclusivamente via AVA (SIGAA), na tarefa correspondente à disciplina.
- Prazo: conforme calendário acadêmico divulgado pelo professor.
- Entregas fora do prazo: desconto de 1 ponto por dia de atraso, até o limite de 5 dias corridos. Após esse limite, a atividade não será aceita.
- Dúvidas técnicas: utilize o fórum da disciplina ou os horários de monitoria.

Este enunciado é completo e suficiente para a realização da atividade. A solução — modelo, scripts, índices e análises — deve ser integralmente construída pelo aluno. Nenhum código gerado por IA, gabarito de terceiros ou script copiado de fontes externas será aceito como entrega.

Bom trabalho.